

How to View My Project? – A Guide

My project, SignLink, is a sign language translation system comprising a native Android mobile application, a wearable hardware device to collect surface electromyography signals (currently a work in progress!), and an integrated AI model.

Project Directory Breakdown

- designs
 - Contains all UI/UX wireframes, user flow diagrams, and the Hierarchical Task Analysis (HTA) mapping the system's foundational interface logic.
- sign_lang_classification
 - The core machine learning sandbox. This folder holds the Python scripts, Jupyter notebooks, and signal preprocessing pipelines used to evaluate and train the gesture classification models.
- android_app
 - The complete native Android application source code developed in Kotlin using Android Studio. This includes the UI workflows (Welcome, Pairing, Calibration, and Home modules) and the Bluetooth Low Energy (BLE) peripheral handling architecture.
- datasets
 - Stores the baseline surface electromyography data streams including the publicly available Myo Armband dataset records utilized for offline validation and training phase simulations.
- grading_and_submissions
 - Dedicated archive containing academic project documentation, evaluation score sheets, progress reports, and milestone materials submitted for assessment.

How to View and Run Each Section

1. Android Application

- Viewing the Code: Access the **android_app** folder directly within this directory to browse the Kotlin source code, layout XML files, and project configuration assets.
- Running the Application: Because the application was built natively, running the project either on a physical Android smartphone or a virtual emulator on your laptop requires importing the entire android_app folder directly into Android Studio as an existing project and building it via Gradle.

2. AI Model Pipeline

- Opening the Workspace: Open the **sign_lang_classification** folder inside VS Code (or your preferred Python IDE). The primary file to review and execute is **RandomForestClassifier.ipynb**.
- Setting Up the Environment: To run the notebook blocks, you must use a virtual Python environment to handle dependencies. If you haven't set one up yet, follow the step-by-step guide provided in the **steps.md** file located directly inside the sign_lang_classification folder. Once the environment is created and activated, you can execute the notebook blocks sequentially to test the model pipeline.

3. Wearable Hardware

- Hardware Evaluation: The physical wearable component is not included for direct testing. However, detailed schematic diagrams, component configurations, and assembly architectures are fully documented within the main report to allow for replication.